

COPYRIGHT PROJECT REPORT
On
ONLINE EXAMINATION AND EVALUATION
PLATFORM

COMPUTER SCIENCE AND ENGINEERING
B.E. Batch-2022
in
APRIL-2026



Under the Guidance of :	Submitted By:
<i>Dr. Monika Aggarwal</i>	<i>Rudrakshi-2210990747</i>
<i>Assistant Professor</i>	<i>Rishab Pahwa-2210990724</i>
	<i>Samarth Sehdev-2210990773</i>
	<i>Shikhar Gussain-2210990820</i>

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY
PUNJAB

ABSTRACT

Exam Matrix is a full-stack web-based application designed to modernize and streamline the examination and evaluation process for educational institutions and training platforms. This system supports a variety of question formats, including multiple-choice questions (MCQs), and checkbox questions providing flexibility to educators while catering to diverse testing needs. It features a robust and user-friendly dashboard tailored for students, faculty, and administrators, offering real-time insights and easy navigation.

The platform includes a dynamic calendar interface to manage and display upcoming tests, schedules, and deadlines, ensuring students stay informed and prepared. For administrators, Exam Matrix offers advanced analytics and data-driven dashboards that highlight top performers, track test performance over time, and support informed decision-making. The system allows role-based access, secure login, and the ability to schedule tests, generate results, and monitor student progress efficiently.

Designed with scalability, usability, and security in mind, *Exam Matrix* is a comprehensive solution for conducting secure, interactive, and efficient online examinations and evaluations.

INDEX

S.No.	Title	Page Number
1.	Introduction	4
2.	S/W , H/W Requirements	4-6
3.	Database Analysis	6,7
4.	Project Snapshots	8-21
5.	Code Snapshots	22 – 26
6.	Conclusion	26
7.	Future Scope	27
8.	References	28

1. INTRODUCTION

1.1 Background

In today's digital era, the need for efficient and secure online examination systems has grown tremendously, especially in the wake of global shifts toward remote learning and virtual assessments. Traditional methods of conducting exams are not only time-consuming and resource-heavy but also vulnerable to malpractices and inefficiencies in evaluation. To meet the growing demand for scalable and reliable examination processes, technology-driven solutions have become essential. *Exam Matrix* emerges as a modern examination and evaluation platform that bridges the gap between institutions and learners. It enables educational organizations and other exam-conducting bodies to administer exams securely, assess results instantly, and streamline the overall examination lifecycle from registration to results analysis.

1.2 Problem Statement

Despite technological advancements, many examination systems still lack flexibility, interactivity, and automated evaluation mechanisms. Some of the prevailing issues in traditional and outdated systems include:

- Manual paper-checking and result processing delays.
- Inability to support diverse question formats such as coding questions.
- Lack of features for student performance tracking and progress visualization.
- Limited integration of time management and scheduling tools.
- Security concerns related to data integrity and cheating prevention.

There is a critical need for a unified platform that supports varied examination types, ensures data confidentiality, offers real-time analysis, and caters to both students and administrators with an intuitive interface.

2. Software and Hardware Requirement Specification

2.1 Methods

Exam Matrix was developed following key software engineering practices to ensure scalability, maintainability, and user-centric performance:

- Agile Development Methodology: Iterative development cycles to implement, test, and refine features.

- Component-Based Architecture: Using React.js[1] for modular and reusable frontend components.
- API-Driven Design: RESTful APIs built with Express.js [5] for seamless frontend-backend communication.
- ORM Integration: Prisma ORM [6] ensures efficient, type-safe interactions with the NeonDB [9] database.
- Security-First Development: Features such as JWT [8] -based authentication and browser activity monitoring are implemented to maintain data integrity and prevent cheating.

2.2 Programming/Working Environment

- Frontend Development:
 - Framework: React.js[1]
 - Styling: CSS [2]
 - Data Management: TanStack React Query [3] for caching and managing server state.
 - Development Tools: Visual Studio Code[9], Git
- Backend Development:
 - Server Framework: Node.js [4] with Express.js[5]
 - ORM: Prisma ORM [6]
 - Database: PostgreSQL [7]
 - Authentication: JSON Web Tokens (JWT)[8]
 - Cloud Database: Neon DB (cloud-native PostgreSQL) [9]
- Version Control and Deployment:
 - Version Control: Git and GitHub
 - Deployment Platform: Complete full stack project hosted on render

2.3 Requirements to Run the Application

- Software Requirements:
 - Operating System: Windows / macOS / Linux
 - Browser: Latest versions of Chrome, Firefox, Brave, Safari, or Edge
 - Node.js: v16 or above [4]
 - NPM (Node Package Manager): v8 or above

- Hardware Requirements:
 - Processor: Intel i5 or higher or equivalent
 - RAM: Minimum 8 GB (recommended 16 GB for development)
 - Storage: At least 1 GB of free disk space
 - Internet: Stable broadband connection for deployment and access

3. Database Analysing , Design and Implementation

3.1 Database Overview

The database for *Exam Matrix* is designed using a relational schema powered by PostgreSQL [7], hosted on Neon DB [9], a cloud-native, scalable database solution. Neon DB [9] enables seamless cloud deployment, ensuring high availability, low-latency data access, and automatic scaling. This architecture ensures reliable, secure, and efficient management of exam-related data such as user accounts, test records, questions, results, and analytics.

3.2 Database Design

The core architecture includes the following primary entities:

- User: Represents both students and administrators with links to their profiles and associated actions.
- Profile: Stores extended user information and preferences.
- Class: Created by administrators to group students and assign tests.
- Test: Represents an individual examination with a predefined structure.
- Question: Each test contains multiple questions that can be of various types.
- Option: Linked to multiple-choice questions, storing possible answers.
- Answer: Stores the correct or selected answers based on question type.
- TestAttempt: Logs each user's test submissions and related data.
- TestClass: A mapping table linking tests to classes.
- Enrollment: Tracks which users are enrolled in which classes.

3.3 Key Relationships

- A User can create multiple Classes and Tests.
- A Class can have many Users (via Enrollment).

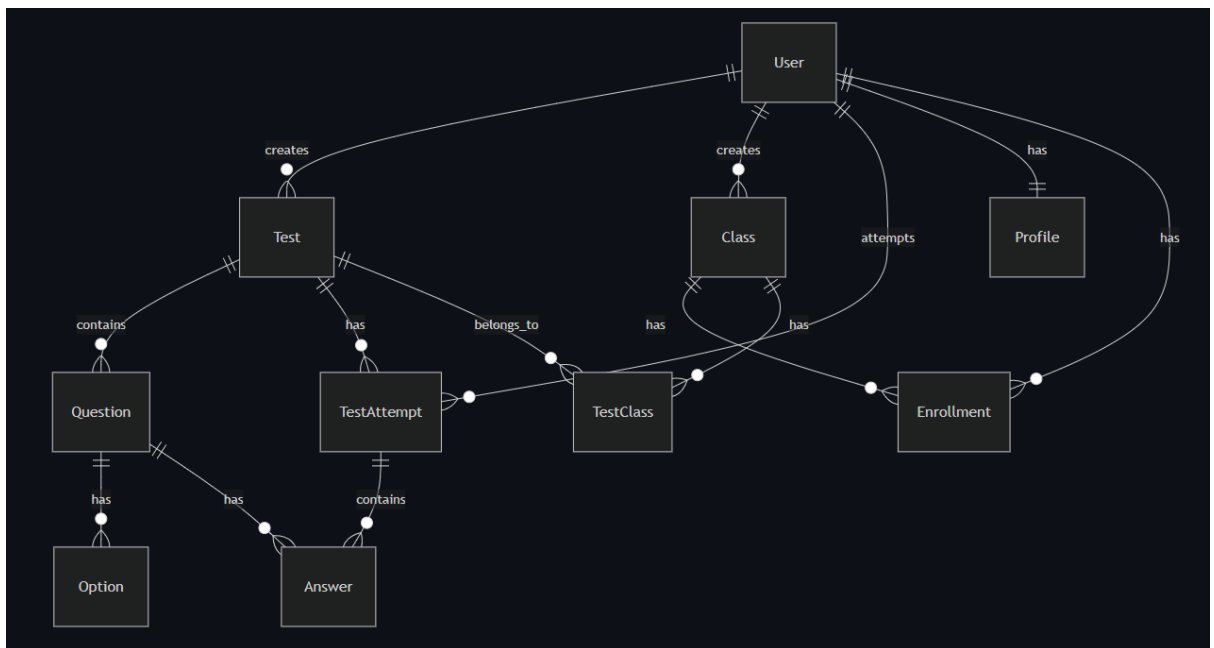
- A Test belongs to a class through the TestClass entity.
- A Test contains many Questions, which in turn have Options and Answers.
- A User can attempt many Tests, which is recorded in TestAttempt.
- Each TestAttempt stores selected answers for evaluation and analytics.

3.4 Data Relationships Summary

The ER diagram shown below visualizes the interrelation among the core entities.

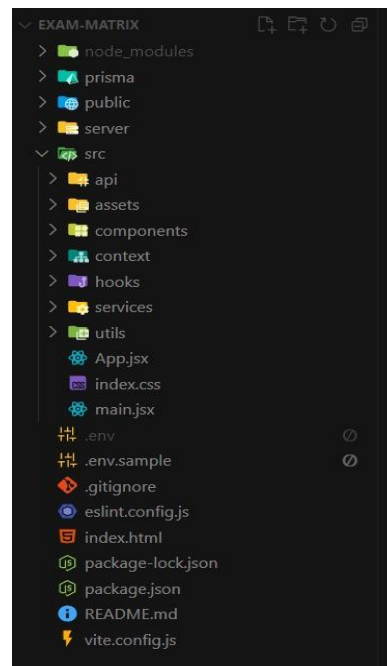
It clearly represents one-to-many and many-to-many relationships such as:

- One User $\leftrightarrow$ Many Tests
- One Test $\leftrightarrow$ Many Questions
- One Question $\leftrightarrow$ Many Options
- Many Users $\leftrightarrow$ Many Classes (via Enrollment)
- One User $\leftrightarrow$ Many TestAttempts

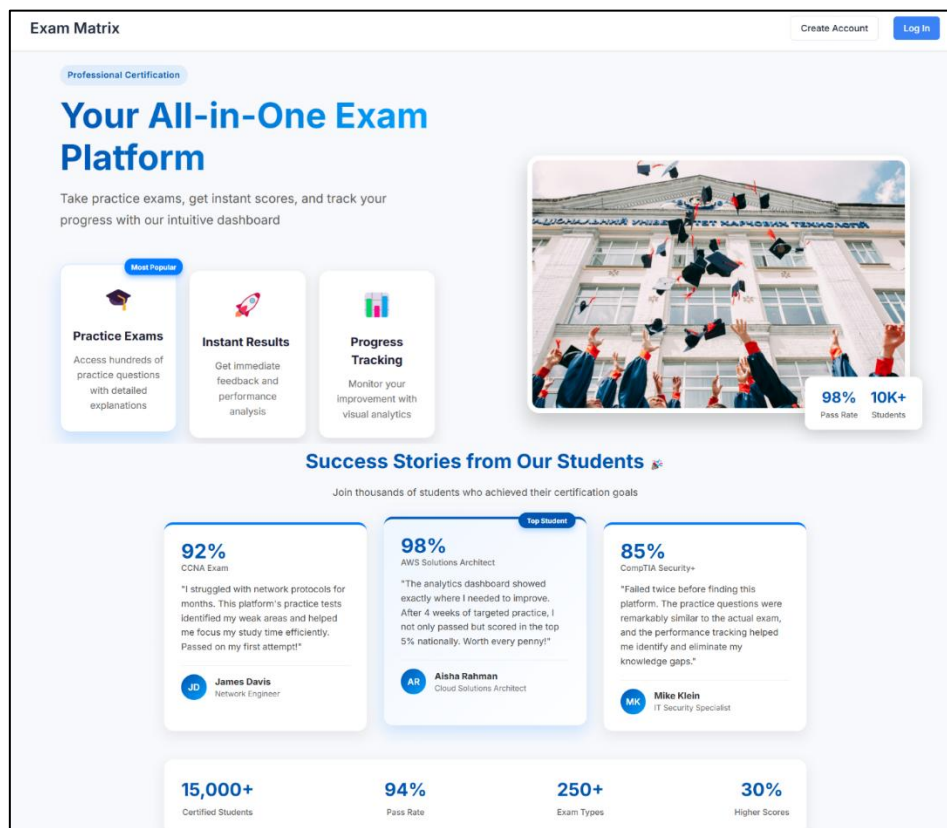


4. Program's Structure Analysing and GUI Constructing

4.1 Project Directory Structure:



4.2 GUI Snapshots:



Home Page of Exam Matrix:

Clean and modern interface with easy navigation, a welcome message, and quick access to login or upcoming exams.

Exam Matrix

Sign in to your account

Email

Password

[Show](#)

☐ Remember me [Forgot Password?](#)

[Sign In](#)

Don't have an account? [Create account](#)

Need help? [Contact Support](#)

Create Your Account

Please fill in the details to get started

First Name

Last Name

Email Address

Password [Show](#)

Confirm Password

Select Your Role

☒ **Student**

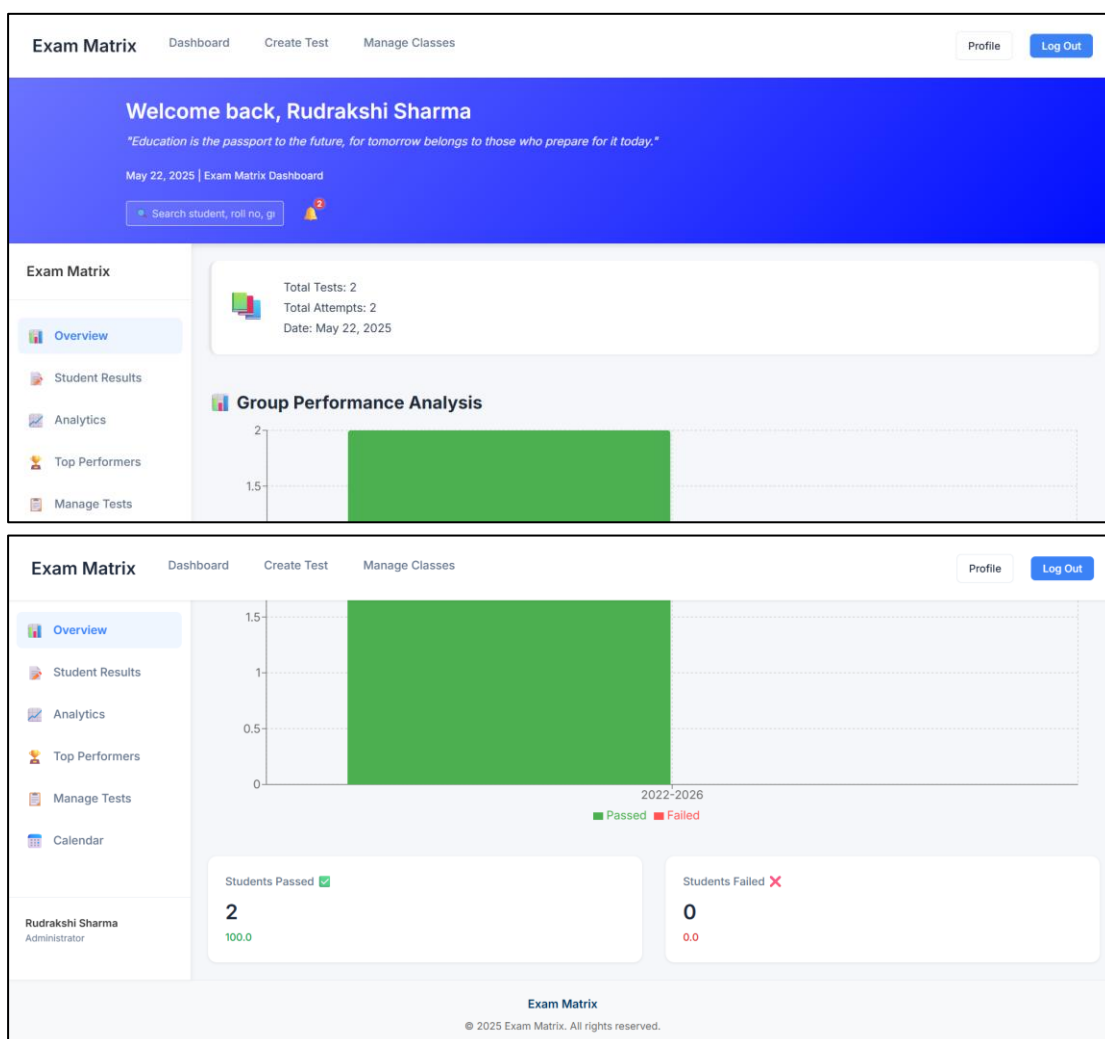
Attempt exams and view results

☐ **Admin**

Manage exams, questions, and user records

Login and Create Account Page:

Allows users to sign in or register with role-based access for Admin and Student.



Admin Dashboard (Group Performance Analysis):

Shows detailed charts and metrics to track group exam results and progress.

Exam Matrix

Overview

Student Results

Analytics

Top Performers

Manage Tests

Calendar

All Students Results

All Groups

All Grades

Sort by Marks

Sno.	Name	Roll Number	Group	Test	Marks	Grade	Status	Actions
1	Sakshi Rana	2210990769	2022-2026	22-May	100/100	A	PASSED	View Recording
2	Sakshi Rana	2210990769	2022-2026	20-May	67/100	D	PASSED	View Recording

Admin Dashboard (Student Results):
Displays results of all students with sortable tables and summary statistics.

Exam Matrix

Overview

Student Results

Analytics

Top Performers

Manage Tests

Calendar

Rudrakshi Sharma

Administrator

Advanced Analytics

Performance Summary

100.0% overall pass rate across all tests

Pass rate is above target of 70%

Test Statistics

2 total tests created

2 total test attempts

Student Engagement

1 unique students took tests

2 tests per student on average

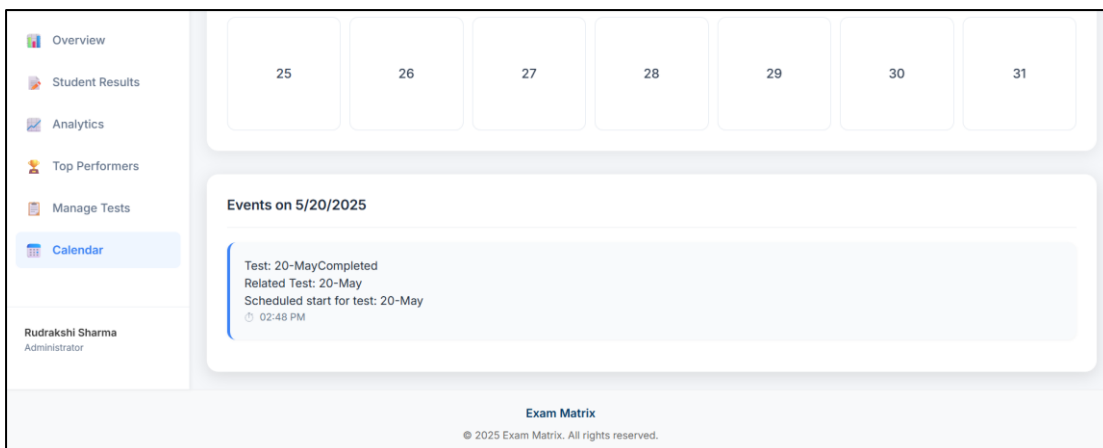
Score Distribution

90-100%		1 students (50%)
80-89%		0 students (0%)
70-79%		0 students (0%)
60-69%		1 students (50%)
50-59%		0 students (0%)
40-49%		0 students (0%)
Below 40%		0 students (0%)

Admin Dashboard (Advanced Analytics):
Provides in-depth data insights with interactive charts and trend analysis tools.

Exam Matrix	Top Performers	
- Overview - Student Results - Analytics Top Performers - Manage Tests - Calendar	Sakshi Rana Roll No: 2210990769 Group: 2022-2026 100% Sakshi Rana Roll No: 2210990769 Group: 2022-2026 67% Performance Insights Test Participation 2 total test attempts recorded Group Performance 1 groups participated in tests Average Score 83% average score	

Admin Dashboard (Top Performers):
Highlights highest-scoring students with rankings and performance summaries.



Calendar Interface (Admin):

Interactive scheduler for managing exams and important dates efficiently.

Create Test:

Enables admins to design exams by adding questions, setting time limits, and choosing formats.

Test Settings:

Allows customization of test parameters like duration, access controls, and maximum attempts

Questions

Question 1

Multiple Choice
☒ Required
Duplicate
Remove

Question Text *

What will be the output of the following C++ code?

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    cout << a + 3;
    return 0;
}
```

Options *

☐ 53
Remove

☒ 8
Remove

☐ a+3
Remove

☐ Compilation Error
Remove

Add Option

Create Test MCQ Question:

Interface to add multiple-choice questions with options and correct answer selection.

Question 2

Checkbox
☒ Required
Duplicate
Remove

Question Text *

Q2. Which of the following are valid variable names in C++?

Options *

☐ my-variable
Remove

☒ _counter
Remove

☐ 2ndValue
Remove

☒ value_2
Remove

Add Option

Add Multiple Choice Question
Add Checkbox Question
Add Coding Question

Create Test Checkbox Question:

Interface to add multiple-answer questions where multiple options can be marked correct.

Test Preview (Admin View)

Back to Editor
Save Test

Testing of 22 May

(No description)

Duration: 60 minutes
Attempts allowed: 1
Expires: Never
Available from: 5/22/2025, 6:45:00 PM

Question 1 * (MCQ)

What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5; cout << a + 3; return 0; }

☐ 53
☒ 8 ✓
☐ a+3
☐ Compilation Error

Question 2 * (CHECKBOX)
Q2. Which of the following are valid variable names in C++?

☐ my-variable

☒ _counter ✓

☐ 2ndValue

☒ value_2 ✓

Test Preview (Admin):

Allows admins to review the full exam layout and content before publishing.

Test Preview (Student View)
Back to Editor
Save Test

Testing of 22 May
 (No description)
 Duration: 60 minutes
 Attempts allowed: 1
 Expires: Never
 Available from: 5/22/2025, 6:45:00 PM

Question 1 * (MCQ)
 What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5; cout << a + 3; return 0; }

Question 2 * (CHECKBOX)
 Q2. Which of the following are valid variable names in C++?

Test Preview (Student):

Shows students a sample view of the exam format and question types before starting.

Exam Matrix

- Overview
- Student Results
- Analytics
- Top Performers
- Manage Tests**
- Calendar

Manage Tests
+ Create New Test

Testing of 22 May
 60 mins 2 questions 1 attempts
 No expiry Starts: 5/22/2025 Inactive
 No description provided
 Settings

22-May
 60 mins 4 questions 1 attempts
 No expiry Starts: 5/22/2025 Complete
 No description provided
 Settings

20-May
 60 mins 3 questions 1 attempts
 No expiry Starts: 5/20/2025 Complete
 No description provided
 Settings

Manage Test:

Displays a list of tests with options to activate, disable, publish, and configure recording settings.

Create Class:

Allows admins to set up a new class with details and enrollment options.

← Manage Enrollments: Final Testing

ENROLLED STUDENTS **PENDING REQUESTS 1**

Student	Email	Roll Number	Class	Batch	Requested On	Actions
Sakshi Rana	sakshi769.be22@chitkara.edu.in	2210990769	G-12	2022-2026	5/22/2025	

Class Information

Class Code
7BC226

Share this code with students to allow them to join the class.

← BACK TO CLASS DETAILS

Pending Requests:

Shows student join requests awaiting approval to enter the class.

← Manage Enrollments: Final Testing

ENROLLED STUDENTS PENDING REQUESTS

Student	Email	Roll Number	Class	Batch	Joined On
 Sakshi Rana	sakshi769.bc22@chitkara.edu.in	2210990769	G-12	2022-2026	5/22/2025

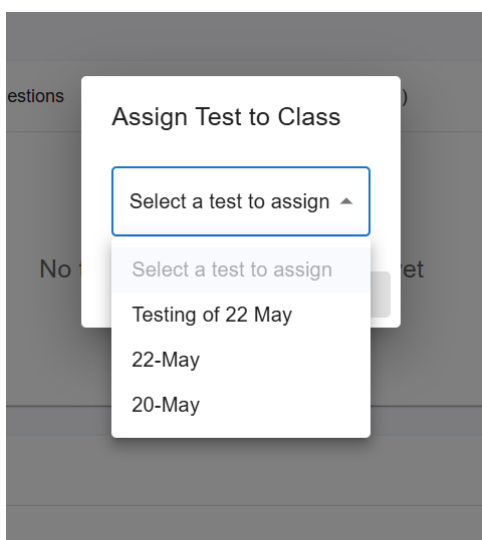
Class Information

Class Code
7BC226

Share this code with students to allow them to join the class.

← BACK TO CLASS DETAILS

Enrolled Students:
Lists all students currently enrolled and active in the class.



Assign Test:
Let Admins allocate exams to specific students or groups.

Admin Profile

Personal Information

First Name
Rudrakshi

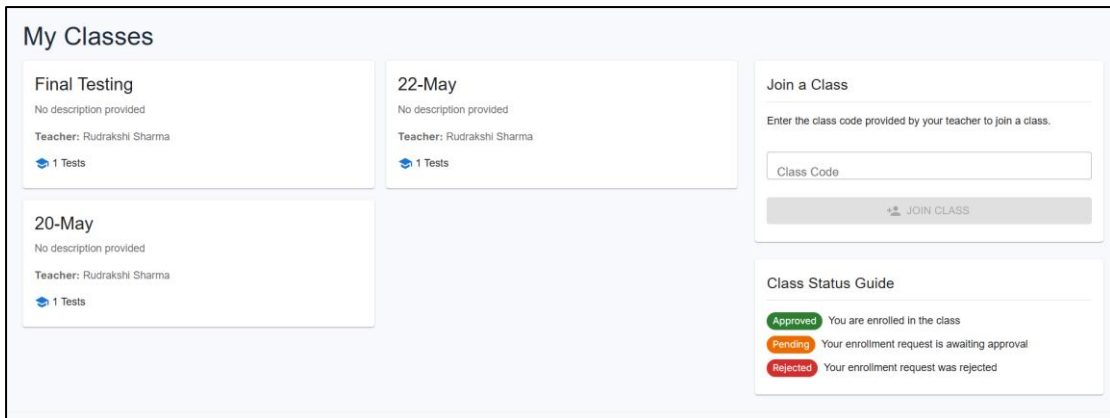
Last Name
Sharma

Email
rudrakshisharma86@gmail.com

Password [Change Password](#)

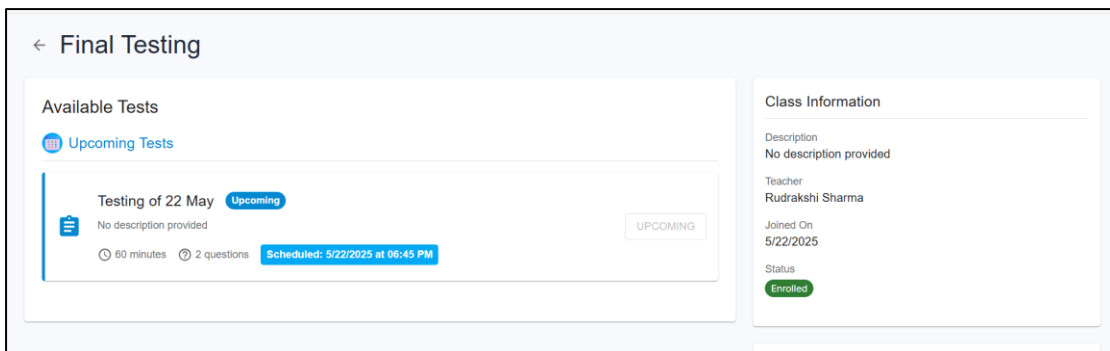
[Save Changes](#)

Admin Profile:
Displays admin's personal information and settings for account management.



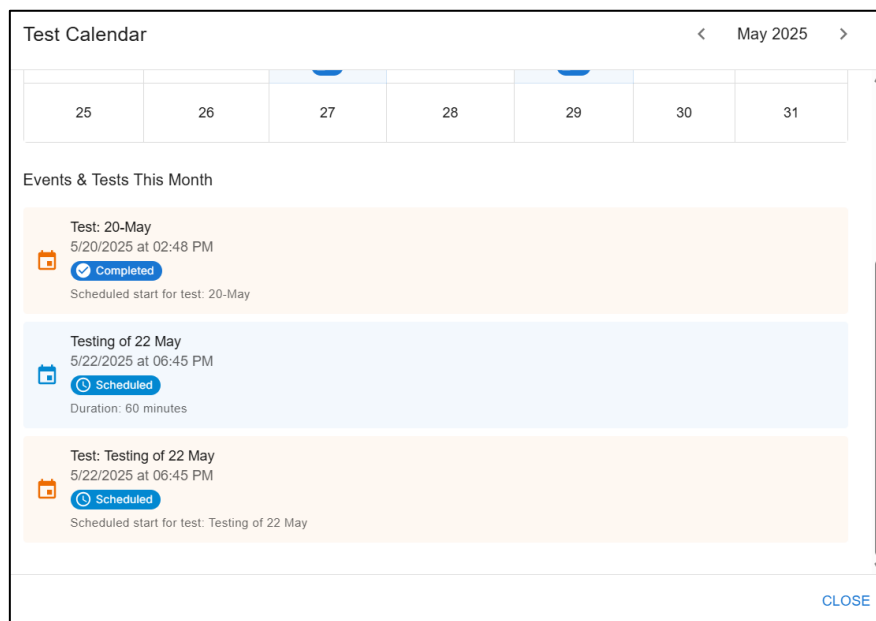
My Classes (Student View):

Displays all the classes the student is currently enrolled in with quick access to details.



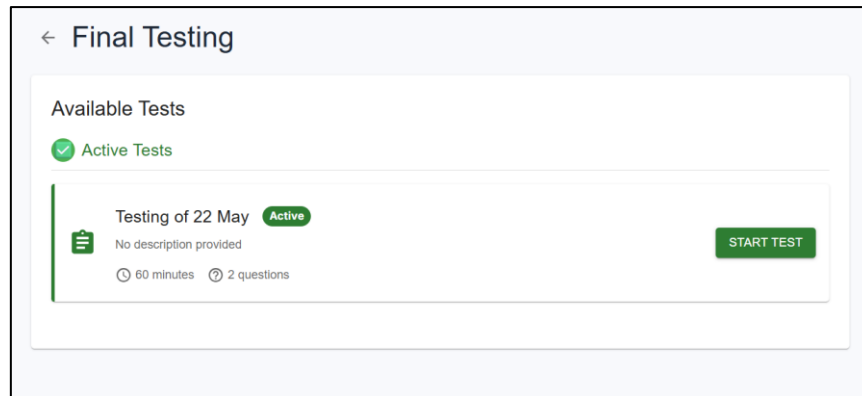
Assigned Classes (Student View):

Displays classes assigned to the student with details like Upcoming tests.

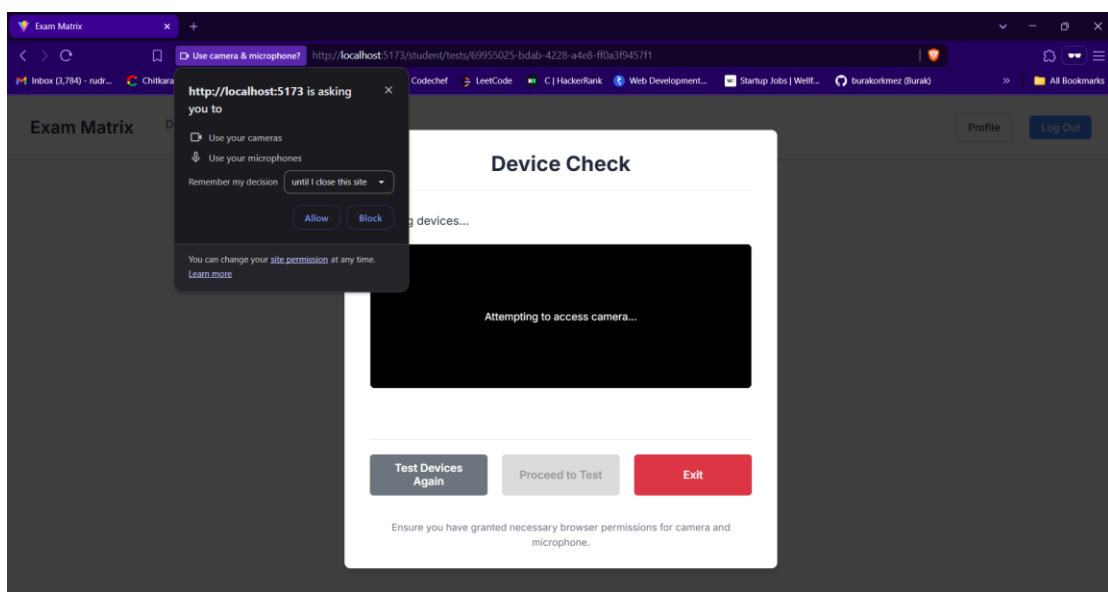


Calendar Interface (Student View):

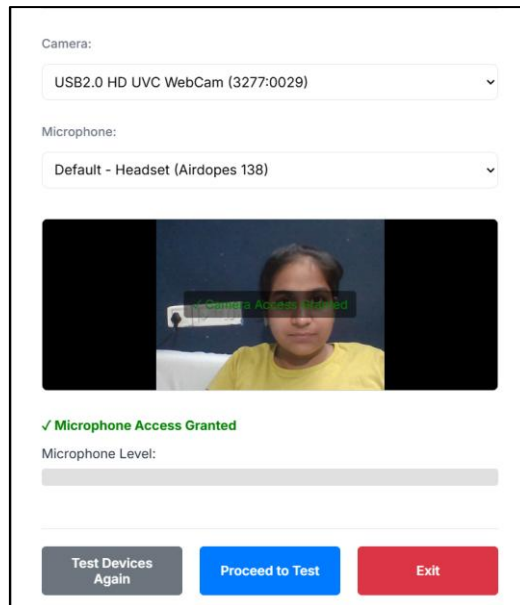
Displays completed and upcoming tests marked on their respective dates for easy tracking.



Available Tests (Student View):
Shows all tests currently accessible for the student to start or continue.

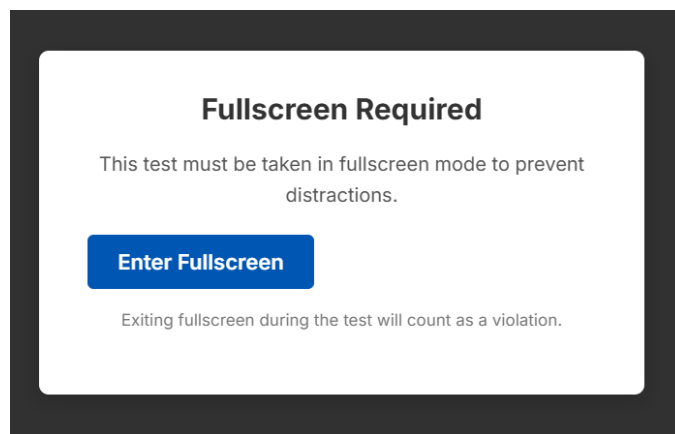


Camera and Microphone Permission:
Prompts the user to allow access to their camera and microphone for monitoring during the exam.



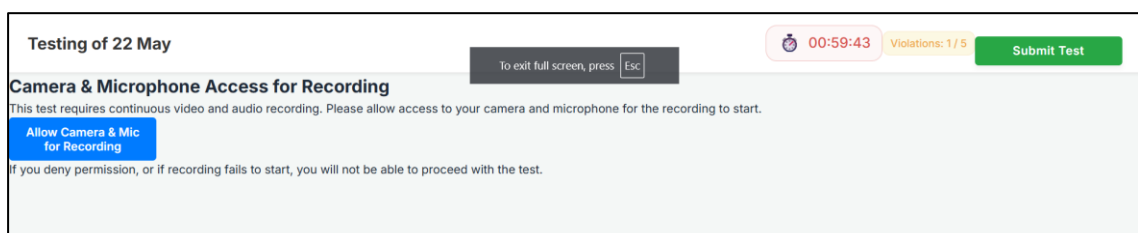
Camera and Microphone Status:

Displays live indicators showing if the camera and microphone are active and recording.



Fullscreen Required:

Informs the user that the exam must be taken in full screen mode to prevent distractions or cheating.



Requesting Camera and Microphone Permission:

Prompts the user again to grant access to their camera and microphone before starting the exam.

Testing of 22 May 00:59:34 Violations: 1 / 5 Submit Test

Question 1 (MCQ)

What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5; cout << a + 3; return 0; }

☐ 53

☒ 8

☐ a+3

☐ Compilation Error

Attempt Test (MCQ):

Allows the student to select one correct option from multiple choices.

Question 2 (CHECKBOX)

Q2. Which of the following are valid variable names in C++?

☐ my-variable

☒ _counter

☒ 2ndValue

☐ value_2

Attempt Test (Checkbox):

Enables the student to select multiple correct answers from the given options.

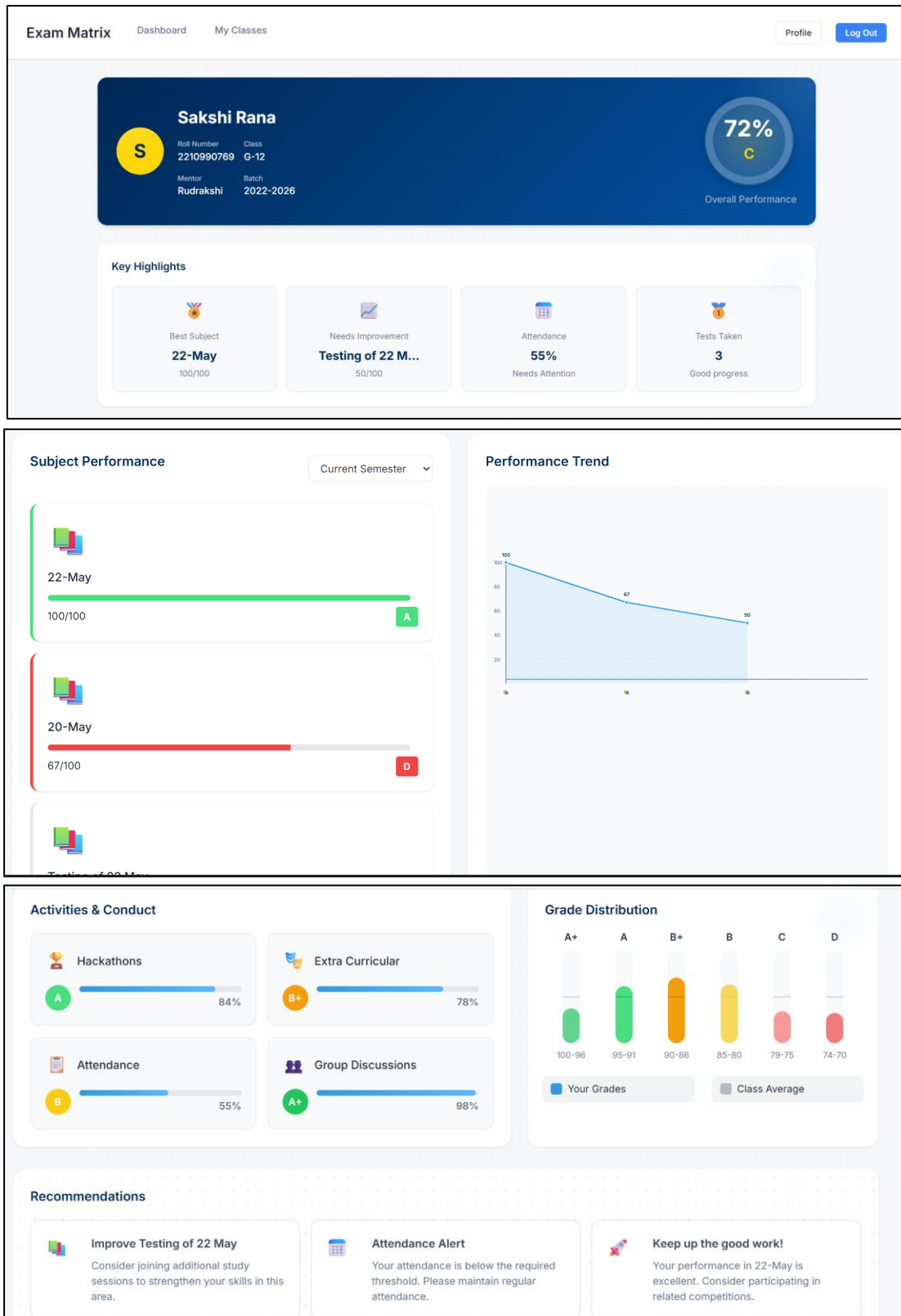
Testing of 22 May 00:59:10 Violations: 3 / 5 Submit Test

Violation Detected (3/5): Switched Tab/Window

☒ 8

Violation Warning:

Alerts the student on actions like Ctrl+C, Ctrl+V, or tab switching — with a maximum of 5 violations allowed.



Student Dashboard:

Displays subject-wise results, highlights best and weak subjects, tracks test attempts, and gives a quick view of overall performance.



Student Profile

Personal Information

First Name

Sakshi

Last Name

Rana

Email

sakshi769.be22@chitkara.edu.in

Academic Information

Roll Number

2210990769

Class/Grade

G-12

Batch/Year

2022-2026

Mentor

Rudrakshi

Password

Change Password

Save Changes

Student Profile:

Shows personal details , like Mentor , Batch details and allows updating account information and preferences.

5. Code Implementation and Database Connections

```
index.js > ...
You, 14 hours ago | 1 author (You)
import express from "express";
import cors from "cors"; 5k (gzipped: 2.1k)
import dotenv from "dotenv"; 6.8k (gzipped: 3k)
import authRoutes from "./routes/auth.js";
import userRoutes from "./routes/users.js";
import testRoutes from "./routes/tests.js";
import classRoutes from "./routes/classes.js";
import calendarRoutes from "./routes/calendar.js";

// Load environment variables
dotenv.config();

const app = express();
const PORT = process.env.PORT || 5000; 'process' is not defined.

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use("/api/auth", authRoutes);
app.use("/api/users", userRoutes);
app.use("/api/tests", testRoutes);
app.use("/api/classes", classRoutes);
app.use("/api/calendar", calendarRoutes);

// Root route
app.get("/", (req, res) => {
  res.send("Exam Matrix API is running");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

main server entry file "index.js"

```
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken"; 48.2k (gzipped: 13.8k)
import { PrismaClient } from "@prisma/client"; 41.9k (gzipped: 15.8k)

const prisma = new PrismaClient();

// Register a new user
> export const register = async (req, res) => { ...
};

// Login user
> export const login = async (req, res) => { ...
};
```

login register Controllers



```
You, 14 hours ago | 1 author (you)
import { PrismaClient } from "@prisma/client"; 41.9k (gzipped: 15.8k)
import bcrypt from "bcrypt";

const prisma = new PrismaClient();

// Get current user with profile data
> export const getCurrentUser = async (req, res) => { ...
};

// Get user profile
> export const getProfile = async (req, res) => { ...
};

// Update user profile
> export const updateProfile = async (req, res) => { ...
};
```

user Controllers

```
You, 14 hours ago | 1 author (you)
import { PrismaClient } from "@prisma/client"; 42.4k (gzipped: 15.9k)
import { Prisma } from "@prisma/client";

const prisma = new PrismaClient();

// Create a new test
> export const createTest = async (req, res) => { ...
};

// Get all tests
> export const getAllTests = async (req, res) => { ...
};

// Get test by ID
> export const getTestById = async (req, res) => { ...
};
```



```
// Update test
> export const updateTest = async (req, res) => { ...
};

// Delete test
> export const deleteTest = async (req, res) => { ...
};

// Publish test
> export const publishTest = async (req, res) => { ...
};

// Start test attempt
> export const startTest = async (req, res) => { ...
};

// Submit test
> export const submitTest = async (req, res) => { ...
};

// Get test results
> export const getTestResults = async (req, res) => { ...
};

// Get all test attempts (admin only)
> export const getAllTestAttempts = async (req, res) => { ...
};

// Get user's own test attempts
> export const getUserTestAttempts = async (req, res) => { ...
};
```

Test Controllers

```
You, 14 hours ago | 1 author (You)
1 import { PrismaClient } from "@prisma/client";      You, 14 hours ago • Remove unused
2 import crypto from "crypto";
3
4 const prisma = new PrismaClient();
5
6 // Create a new class
7 > export const createClass = async (req, res) => { ...
8   };
9
10 // Get all classes (admin sees all their classes, students see classes they're
11   enrolled in)
12 > export const getAllClasses = async (req, res) => { ...
13   };
14
15 // Get class by ID
16 > export const getClassById = async (req, res) => { ...
17   };
18
19 // Update class
20 > export const updateClass = async (req, res) => { ...
21   };
22
23 // Delete class
24 > export const deleteClass = async (req, res) => { ...
25   };
26
```




```
// Generate a new class code
> export const regenerateClassCode = async (req, res) => { ...
};

// Join class (for students)
> export const joinClass = async (req, res) => { ...
};

// Get pending enrollment requests (for admin)
> export const getPendingEnrollments = async (req, res) => { ...
};

// Approve or reject enrollment request
> export const updateEnrollmentStatus = async (req, res) => { ...
};

// Assign test to class
> export const assignTestToClass = async (req, res) => { ...
};

// Remove test from class
> export const removeTestFromClass = async (req, res) => { ...
};

// Get tests for a class
> export const getClassTests = async (req, res) => { ...
};

// Get all classes for a student
> export const getClasses = async (req, res) => { ...
};
```

Class Controllers

```
You, 14 hours ago | 1 author (You)
import { PrismaClient } from "@prisma/client"; 41.9k (gzipped: 15.8k)

const prisma = new PrismaClient();

// Get all calendar events
> export const getAllCalendarEvents = async (req, res) => { ...
};

// Create a new calendar event
> export const createCalendarEvent = async (req, res) => { ...
};

// Update a calendar event
> export const updateCalendarEvent = async (req, res) => { ...
};

// Delete a calendar event
> export const deleteCalendarEvent = async (req, res) => { ...
};
```

Calendar Controllers

```
import jwt from "jsonwebtoken"; 48.2k (gzipped: 13.8k)
import { PrismaClient } from "@prisma/client"; 41.9k (gzipped: 15.8k)
import process from "process";

const prisma = new PrismaClient();

// Authenticate user middleware
> export const authenticate = async (req, res, next) => { ...
};

// Authorize admin middleware
> export const authorizeAdmin = (req, res, next) => { ...
};
```

JWT [8] auth Middleware Functions



```
import React, { createContext, useContext, useState, useEffect } from 'react'; 7.9k (
import { useNavigate } from 'react-router-dom'; 240k (gzipped: 75.5k)
import * as authService from '../api/authService';

// Create the auth context
const AuthContext = createContext();

// Auth provider component
> export const AuthProvider = ({ children }) => { ...
};

// Custom hook to use the auth context
> export const useAuth = () => { Fast refresh only works when a file only exports con
}; You, 2 months ago • Update project structure with new features and ...
```

Frontend Auth Context

```
import axios from 'axios'; 62.6k (gzipped: 23.2k)

const API_URL = 'http://localhost:5000/api';

> const axiosInstance = axios.create({ ...
});

// Add a request interceptor to include the auth token in requests
> axiosInstance.interceptors.request.use(...
);

// Add a response interceptor to handle common errors
> axiosInstance.interceptors.response.use(...
);

export default axiosInstance; You, 2 months ago • Update project structure wit
```

Frontend Central Axios Api service

6. Conclusion

Exam Matrix represents a significant advancement in online examination systems, offering a comprehensive solution that addresses the limitations of traditional assessment methods. By providing a secure, flexible, and analytical platform for conducting examinations, we've created a tool that benefits both educational institutions and students alike.

The system's role-based access, multi-format question support, robust security measures, and detailed analytics dashboard represent a complete transformation of the examination process. Our future enhancements, including expanded question formats, advanced security features, and additional functionality like calendar integration, will further solidify Exam Matrix as a leading solution in educational assessment. As we continue to develop and refine this platform, we remain committed to enhancing the examination experience through innovation, security, and actionable insights. Thank you for your attention, and we welcome any questions or feedback you may have.

7. Future Scope

As online examinations continue to gain momentum, ensuring the integrity of assessments is more crucial than ever. While *Exam Matrix* already incorporates fundamental security features such as tab-switch detection, disabled copy-paste, and single-attempt enforcement, future iterations will emphasize enhanced proctoring and real-time monitoring. Planned improvements include:

- **Multi-language Coding & AI Feedback**

Introducing support for multiple programming languages such as Python, Java, C++, and more will enable students to code in their preferred language. Integrated AI features will provide real-time hints, identify common mistakes, and suggest improvements to help learners write better code more efficiently.

- **AI Proctoring**

Advanced AI-driven proctoring will be implemented to ensure exam integrity. Features such as facial recognition, eye movement tracking, and behavioral analysis will allow for real-time monitoring and automatic flagging of suspicious activity, minimizing the chances of malpractice.

- **Gamification**

A structured rewards system including achievement badges, leaderboards, and milestone-based incentives will be introduced to create a more engaging and competitive learning environment. This system will motivate students to stay active, improve their performance, and participate more consistently.

- **Mobile App**

A dedicated mobile application will be developed for both Android and iOS platforms, allowing students to take exams, check their results, and receive real-time notifications. This enhancement ensures continuous access to the platform, increasing flexibility and supporting learning beyond desktop environments.

These future enhancements will significantly strengthen the credibility of the *Exam Matrix* platform and make it suitable for conducting high-stakes, remotely-proctored examinations in both academic and professional settings.

8. Bibliography/References

- 1) **React.js** – A JavaScript library for building responsive and dynamic user interfaces.
<https://reactjs.org/>
- 2) **CSS** – A stylesheet language used to design and style web pages.
<https://www.w3schools.com/css/>
- 3) **TanStack React Query** – A powerful library for fetching, caching, and updating asynchronous data in React applications.
<https://tanstack.com/query/latest>
- 4) **Node.js** – A JavaScript runtime environment used for building scalable backend applications.
<https://nodejs.org/>
- 5) **Express.js** – A minimal and flexible Node.js web application framework for building RESTful APIs.
<https://expressjs.com/>
- 6) **Prisma ORM** – A type-safe and intuitive ORM for Node.js and TypeScript, enabling efficient database interaction.
<https://www.prisma.io/>
- 7) **PostgreSQL** – A powerful open-source relational database system known for its robustness and performance.
<https://www.postgresql.org/>
- 8) **JWT (JSON Web Token)** – A compact and secure token format used for authentication and data exchange.
<https://jwt.io/>
- 9) **Neon Database** – A fully managed, cloud-native, serverless PostgreSQL solution optimized for modern applications.
<https://neon.tech/>